DB-Git: Complete One-Shot Implementation Plan & Viva Playbook

Polyglot Version Control System & Next.js Web Portal Using PostgreSQL 16, pgvector & Neo4j 5 | PES University DBMS Level 3 Mini-Project (50 Marks)

Team Members: Naveen Prasad M (SRN: PES1UG24AM379) & Nishanth Nag G (SRN: PES1UG24AM381)

1. Executive Overview & 50-Mark Rubric Alignment Matrix

This implementation blueprint consolidates every architectural fix, database script, Next.js module, Antigravity AI prompt strategy, and Viva explanation required to build **DB-Git** in a single continuous execution session. All timeline/date references have been removed so you can execute Stages 1 through 6 back-to-back in one shot while satisfying all 18 course requirements and all 14 components (50 Marks) of the PES University DBMS Level 3 Evaluation Rubric.

PESU Evaluation Component	Marks	Where Implemented	Exact Deliverable & Verification Proof
1. ER Diagram	2	Section 2 (Figure 2)	10-table PostgreSQL ER schema mapped to Neo4j `(:Commit)` & `(:Tag)` graph nodes.
2. Relational Schema & Constraints	3	Stage 2 (`01_schema_ddl.sql`)	Normalized 3NF/BCNF schema with primary keys, foreign keys, cascades, and Data Dictionary.
3. SQL DDL with Constraints	4	Stage 2 (`01_schema_ddl.sql`, `02_views_rbac_rls.sql`)	`CREATE`, `ALTER`, `DROP`, `CREATE VIEW`, and regex/range `CHECK` constraints.
4. NoSQL / Vector DB Creation	4	Stage 2 (`01_constraints_indexes.cypher`)	Neo4j uniqueness constraints & composite index + PostgreSQL `pgvector vector(384)` & HNSW index.
5. Technology Justification	1	Section 6.1	Workload-driven justification for PostgreSQL (ACID/CAS), Neo4j (DAG/LCA), and `pgvector` (Semantic).
6. DML Operations	4	Stage 4 (`/api/staging`, `/api/branches`)	Explicit `INSERT`, `UPDATE`, `DELETE`, and `SELECT` on `staging_index`, `blobs`, and `branches`.
7. Nested/Correlated/Recursive & Joins	4	Stage 4 (`04_analytics_cte_queries.sql`)	Multi-table `JOIN` + `GROUP BY` + `HAVING`, Correlated `EXISTS` subquery, and `WITH RECURSIVE` CTE.
8. Procedure/Function, Trigger, CTE	6	Stage 2 & 4 (`03_triggers_procs.sql`)	PL/pgSQL `fn_create_atomic_commit()`, immutability triggers on

PESU Evaluation Component	Marks	Where Implemented	Exact Deliverable & Verification Proof
			`blobs`/`commits`, and analytical CTEs.
9. NoSQL / Vector Feature & RUD	7	Stage 4 (`/api/graph/lca`, `/api/graph/rud`)	Neo4j Read (DAG + LCA + Aggregation), Update (`SET` CI status/tags), Delete (`DETACH DELETE` GC) + `pgvector`.
10. Next.js Interface	3	Stage 5 (`/src/app/repos/[repoId]/*`)	4 Next.js 14 App Router views: Explorer & Staging, React Flow DAG Studio, Analytics, and RBAC Settings.
11. Backend/API & DB Integration	2	Stage 3 & 4 (`/src/lib/*`, `/src/app/api/*`)	Next.js Route Handlers using `.env.local` pools + Two-Phase Polyglot Commit Coordinator with Rollback.
12. Auth & RBAC in Database	4	Stage 2 & 3 (`02_views_rbac_rls.sql`, `/api/auth`)	`bcrypt` + JWT auth, 2 system roles, 4 repo roles, native PostgreSQL `CREATE ROLE`/`GRANT`, and RLS policy.
13. Documentation & Git/GitHub	2	Stage 1 & 6 (`README.md`, Test Matrix)	Complete specification, 6 verification test cases (`TC-01` to `TC-06`), and dual-branch GitHub repo.
14. Originality, Contribution & Viva	4	Section 1.1 & Section 6	Balanced commits between `PES1UG24AM379` & `PES1UG24AM381` across DB and Next.js + live Viva queries.

1.1 Two-Member Division of Ownership (Naveen vs. Nishanth)

Team Member	Git Branch Name	Database Layer Ownership	Next.js & API Ownership
Naveen Prasad M SRN: `PES1UG24AM379`	`feat/naveen-sql-cas`	PostgreSQL 10-table DDL, `CHECK` constraints, Views, native DB `ROLE`/`GRANT` & RLS policies, Immutability Triggers, and Analytical/Recursive CTE SQL queries.	PostgreSQL pool (`postgres.ts`), JWT/`bcrypt` Auth (`/api/auth`), Repo & RBAC APIs, CAS Staging API (`/api/staging`), File Explorer UI, RBAC Settings UI, and SQL Analytics UI.
Nishanth Nag G SRN: `PES1UG24AM381`	`feat/nishanth-neo4j-dag`	PL/pgSQL `fn_create_atomic_commit()` stored function, `pgvector` HNSW index, Neo4j uniqueness constraints/indexes, and Cypher LCA, Aggregation & RUD queries.	Neo4j driver (`neo4j.ts`), Vector helper, Branch API, Two-Phase Polyglot Commit API (`/api/commits`), Neo4j LCA & RUD APIs, Interactive React Flow DAG UI, and Semantic Search UI.

2. System Architecture, ER Schema & Antigravity AI Quota Strategy

To build the entire system in one sitting on your Jio Antigravity Pro plan without exhausting your premium model quota, strictly partition your AI usage across **Flash High**, **Sonnet 4.6**, and **Opus 4.6** as shown below.

Antigravity Model	Build Share	Exact One-Shot Tasks to Run on This Model
3.8 Flash High (Primary Workhorse)	80% of Project	1. Scaffolding Next.js, `docker-compose.yml`, `.env.local`, and copying all SQL/Cypher files from Section 5. 2. Writing `/src/lib/postgres.ts`, `/src/lib/neo4j.ts`, `/src/lib/auth.ts`, and CRUD API routes (`/api/auth`, `/api/repos`, `/api/staging`, `/api/branches`, `/api/analytics`). 3. Building the Login page, File Explorer & Staging table, RBAC Settings page, Analytics tables, and `seed.ts`.
Sonnet 4.6 (UI & Graph Precision)	15% of Project	1. Building the interactive **React Flow Commit DAG Visualizer** (`/src/app/repos/[repoId]/graph/page.tsx`) with automatic node layout, branch badges, green LCA Merge-Base highlighting, CI status editor, and Garbage Collection trigger.
Opus 4.6 (Reserve for Complex Tx)	5% of Project (1–2 Prompts Max)	1. Building and verifying the **Two-Phase Polyglot Commit Coordinator** (`/src/app/api/commits/route.ts`) that executes PostgreSQL `BEGIN`, constructs the Merkle tree, calls `fn_create_atomic_commit()`, writes the Neo4j DAG node/edges, and executes PostgreSQL `ROLLBACK` if Neo4j fails.

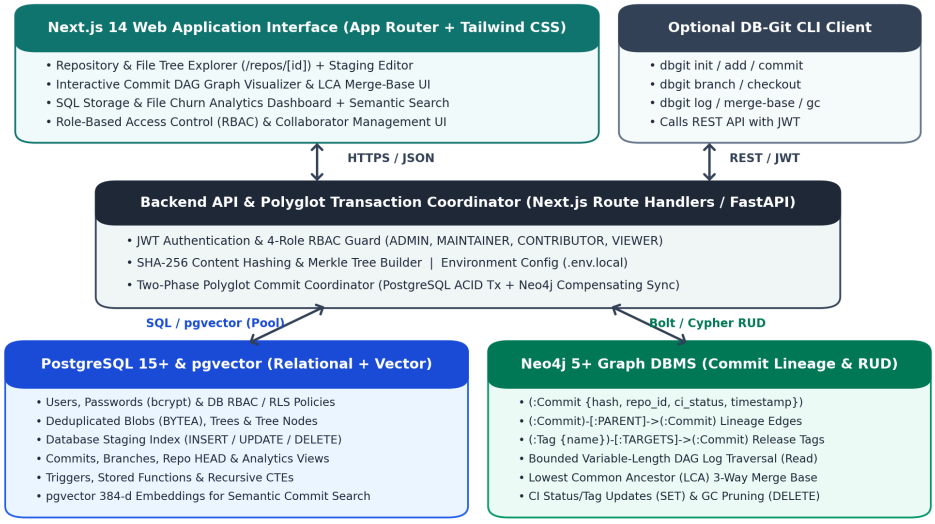


Figure 1: DB-Git Three-Tier Polyglot System Architecture (Next.js 14 App Router + API Coordinator + PostgreSQL/pgvector + Neo4j).

3. Master Project Directory Structure (Where Every File Lives)

Create the project root folder `db-git-polyglot/`. Every script, database file, API route, and UI view maps to the exact paths below:

```
db-git-polyglot/
├── docker-compose.yml                  # Spins up pgvector/pgvector:pg16 (port 5432) +
neo4j:5 (ports 7474, 7687)
├── .env.local                          # Server-side DB credentials & JWT_SECRET (never
committed to Git)
├── .env.example                        # Template showing required environment keys (Rubric
#11)
├── README.md                           # Architecture summary & one-command startup guide
(Rubric #13)
```

```
├── database/
│   ├── postgres/
│   │   ├── 01_schema_ddl.sql              # 10 normalized tables + PK/FK + regex/range CHECK
constraints
│   │   ├── 02_views_rbac_rls.sql          # ALTER, DROP, Views, native PostgreSQL ROLES/GRANTS
& RLS policy
│   │   ├── 03_triggers_procs.sql          # Immutability triggers + fn_create_atomic_commit()
stored function
│   │   └── 04_analytics_cte_queries.sql   # JOIN/GROUP BY/HAVING, Correlated Subquery, WITH
RECURSIVE & LAG() CTE
│   ├── neo4j/
│   │   ├── 01_constraints_indexes.cypher  # Uniqueness constraints on (:Commit) and (:Tag) +
composite index
│   │   └── 02_dag_lca_rud_queries.cypher  # Bounded DAG Read, Safe LCA Merge-Base, Graph
Aggregation, SET & GC DELETE
│   └── seed.ts                            # One-shot seed script populating 4 roles, duplicate
blobs & merge DAG
└── src/
    ├── lib/
    │   ├── postgres.ts                    # PostgreSQL pg.Pool singleton reading
process.env.DATABASE_URL
    │   ├── neo4j.ts                       # Neo4j Bolt driver singleton reading
process.env.NEO4J_URI
    │   ├── auth.ts                        # bcrypt password hashing, JWT token signing & role
verification
    │   └── embeddings.ts                  # Deterministic 384-d text vector generator for
commit messages
    └── app/
        ├── api/
        │   ├── auth/route.ts              # POST register / login with bcrypt & JWT cookie
        │   ├── repos/route.ts             # GET/POST repositories + initial OWNER role & main
branch creation
        │   ├── collaborators/route.ts     # POST/PUT/DELETE repository RBAC roles
(OWNER/MAINTAINER/CONTRIBUTOR/VIEWER)
        │   ├── staging/route.ts           # POST/PUT/DELETE staging_index + SHA-256 content-
addressable blob insert
        │   ├── commits/route.ts           # POST Two-Phase Polyglot Commit (Postgres ACID Tx +
Neo4j MERGE + Rollback)
        │   ├── branches/route.ts          # POST/PUT/DELETE branches & repo_head checkout
switching
        │   ├── graph/lca/route.ts         # GET Neo4j DAG lineage, 3-Way Merge Base (LCA) &
Graph Aggregation
        │   ├── graph/rud/route.ts         # PUT CI status / release tag (SET) & DELETE orphan
commits (dbgit gc)
        │   └── analytics/route.ts         # GET SQL Deduplication View, High-Churn HAVING,
Subquery, Recursive CTE & pgvector
        ├── login/page.tsx                 # Authentication & Role Registration UI
        └── repos/[repoId]/
            ├── page.tsx                   # View 1: Repository File Explorer, Blob Viewer &
Staging Studio
            ├── graph/page.tsx             # View 2: Interactive React Flow Commit DAG, LCA
Calculator & RUD Controls
            ├── analytics/page.tsx         # View 3: SQL Storage Efficiency, High-Churn,
Recursive CTE & Semantic Search UI
            └── settings/page.tsx          # View 4: 4-Role RBAC Collaborator Management
Console
```

4. Step-by-Step One-Shot Execution Plan (Stage 1 to Stage 6)

Follow Stages 1 through 6 sequentially in a single session. Even when building in one shot on one machine, commit Naveen's files under `feat/naveen-sql-cas` (with `git config user.name "Naveen Prasad M"`) and Nishanth's files under `feat/nishanth-neo4j-dag` (with `git config user.name "Nishanth Nag G"`) before merging both into `main` so GitHub Insights displays equal contributions for Rubric #13 and #14.

Stage 1: Workspace Initialization, Docker Compose & Environment Setup

- **Where:** Root directory `db-git-polyglot/`
- **Model to Use:** Terminal / **Flash High**
- **What to Execute:**
  1. Initialize Next.js: `npx create-next-app@latest db-git-polyglot --typescript --tailwind --app --src-dir --eslint`
  2. Install packages: `npm install pg neo4j-driver bcryptjs jsonwebtoken @xyflow/react lucide-react && npm install -D @types/pg @types/bcryptjs @types/jsonwebtoken tsx`
  3. Create `/docker-compose.yml` with two containers:

```
version: '3.8'
services:
  postgres:
    image: pgvector/pgvector:pg16
    container_name: dbgit_postgres
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: dbgit_password
      POSTGRES_DB: dbgit
    ports:
      - "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data
  neo4j:
    image: neo4j:5
    container_name: dbgit_neo4j
    environment:
      NEO4J_AUTH: neo4j/dbgit_neo4j_pass
    ports:
      - "7474:7474"
      - "7687:7687"
    volumes:
      - neo4jdata:/data
volumes:
  pgdata:
  neo4jdata:
```

  4. Create `/.env.local` (and copy keys with dummy values to `/.env.example`):

```
DATABASE_URL=postgresql://postgres:dbgit_password@localhost:5432/dbgit
NEO4J_URI=bolt://localhost:7687
NEO4J_USER=neo4j
NEO4J_PASSWORD=dbgit_neo4j_pass
JWT_SECRET=pesu_dbms_level3_polyglot_secret_2026
```

  5. Start containers: `docker compose up -d` and verify `docker ps` shows both containers healthy.

Stage 2: Database Schema, Constraints, RBAC, Triggers & Stored Functions

- **Where:** `/database/postgres/` and `/database/neo4j/`
- **Model to Use:** Direct Copy-Paste from Section 5 of this PDF (Zero AI tokens needed)
- **What to Execute:**
  1. **Naveen (`PES1UG24AM379`):** Save Section 5.1 into `/database/postgres/01_schema_ddl.sql` and Section 5.2 into `/database/postgres/02_views_rbac_rls.sql`. Execute both against `dbgit_postgres`:

```
docker exec -i dbgit_postgres psql -U postgres -d dbgit <
database/postgres/01_schema_ddl.sql
docker exec -i dbgit_postgres psql -U postgres -d dbgit <
database/postgres/02_views_rbac_rls.sql
```

2. **Nishanth (PES1UG24AM381):** Save Section 5.3 into `/database/postgres/03_triggers_procs.sql` and run it in PostgreSQL. Then open `http://localhost:7474` (or use `cypher-shell`) and execute the constraints and indexes from Section 5.5 (`/database/neo4j/01_constraints_indexes.cypher`).

Stage 3: Database Connection Pools, Auth, RBAC & Branch Lifecycle APIs

- **Where:** `/src/lib/*` and `/src/app/api/{auth,repos,collaborators,branches}/route.ts`
- **Model to Use: Flash High**
- **What to Build:**

File Path	Owner	Implementation Specification for Flash High
`/src/lib/postgres.ts`	Naveen	Export a singleton `Pool` from `pg` using `process.env.DATABASE_URL`, plus a helper `query(text, params)` and `getClient()` for transactions.
`/src/lib/neo4j.ts`	Nishanth	Export a singleton Neo4j driver via `neo4j.driver(process.env.NEO4J_URI!, neo4j.auth.basic(process.env.NEO4J_USER!, process.env.NEO4J_PASSWORD!))`.
`/src/lib/auth.ts` & `/src/lib/embeddings.ts`	Both	`auth.ts`: `bcrypt.hash(pw, 12)`, `bcrypt.compare()`, JWT sign/verify. `embeddings.ts`: Deterministic 384-float normalized embedding generator from commit message tokens so semantic search works offline without external API keys.
`/src/app/api/auth/route.ts`	Naveen	`POST` handler supporting `action: 'register' | 'login'`. Inserts into `users (username, email, password_hash, system_role)` and returns HTTP-only JWT cookie + user JSON.
`/src/app/api/repos/route.ts` & `/api/collaborators/route.ts`	Naveen	Creates repository in `repositories`, inserts creator as `'OWNER'` in `repo_collaborators`, creates default `'main'` branch in `branches`, and initializes `repo_head`. Collaborators route manages `OWNER`, `MAINTAINER`, `CONTRIBUTOR`, and `VIEWER` roles.
`/src/app/api/branches/route.ts`	Nishanth	`POST`: Creates new branch copying current branch's `target_commit_hash`. `PUT`: Switches active checkout branch in `repo_head`. `DELETE`: Deletes non-default branch (`DELETE FROM branches WHERE branch_id = $1 AND is_default = FALSE`).

Stage 4: Core Polyglot Engine (CAS Staging, Two-Phase Commit, LCA & NoSQL RUD)

- **Where:** `/src/app/api/{staging,commits,graph,analytics}/`
- **Model to Use: Flash High** for Staging & Analytics; **Opus 4.6 (1 Prompt)** for Commits & Graph RUD
- **What to Build:**

API Endpoint	Owner & Model	Exact Database Operations Executed
`/src/app/api/staging/route.ts`	Naveen (Flash High)	**POST (Stage File):** Computes `blob_hash = SHA256(content)`. Runs `INSERT INTO blobs ... ON CONFLICT DO NOTHING` (CAS deduplication), then upserts `staging_index`. **PUT (Modify Mode):** Runs `UPDATE staging_index`

API Endpoint	Owner & Model	Exact Database Operations Executed
		`SET file_mode = $1 WHERE ...` **DELETE (Unstage):** Runs `DELETE FROM staging_index WHERE repo_id = $1 AND user_id = $2 AND file_path = $3`.
`/src/app/api/commits/route.ts`	Nishanth (**Opus 4.6**)	**Two-Phase Polyglot Commit:** 1. Reads user's `staging_index` rows + parent commit's `tree_nodes`. 2. Computes Merkle `tree_hash`, `commit_hash`, and 384-d `msg_embedding`. 3. Opens Postgres client: `BEGIN` → inserts `trees` & `tree_nodes` → calls `SELECT fn_create_atomic_commit(...)`. 4. Opens Neo4j session: runs Cypher `MERGE (c:Commit {hash: $commit_hash})... MERGE (c)-[:PARENT]->(parent)`. 5. If Neo4j succeeds → Postgres `COMMIT`. If Neo4j throws → Postgres `ROLLBACK` (zero orphan state).
`/src/app/api/graph/lca/route.ts`	Nishanth (Flash High)	Runs the 3 Neo4j Read queries in Section 5.5B: (1) Bounded DAG history `[:PARENT*0..50]`, (2) Safe 3-Way Merge Base LCA between `branchA` and `branchB`, and (3) Graph Aggregation counting Root, Standard, and Merge commits.
`/src/app/api/graph/rud/route.ts`	Nishanth (Flash High)	**PUT (NoSQL Update):** Runs Cypher `SET c.ci_status = $status, c.verified_at = timestamp()` or creates/retargets a `(:Tag)-[:TARGETS]->(:Commit)` node. **DELETE (NoSQL Delete / GC):** Deletes a tag or runs `dbgit gc` (Section 5.5D) to `DETACH DELETE` unreachable orphan `(:Commit)` nodes.
`/src/app/api/analytics/route.ts`	Naveen (Flash High)	Executes all 5 analytical queries from Section 5.4: (1) `v_repo_storage_efficiency` view, (2) High-Churn `JOIN/GROUP BY/HAVING`, (3) Correlated `EXISTS` + Nested `AVG` subquery, (4) `WITH RECURSIVE commit_ancestry_cte` + `LAG()` delta CTE, and (5) `pgvector` cosine similarity search (`<=>`).

Stage 5: Next.js 14 Web Portal & Interactive React Flow DAG Visualizer

- **Where:** `/src/app/repos/[repoId]/*`
- **Model to Use: Flash High** for Views 1, 3, 4; **Sonnet 4.6** for View 2 (DAG Visualizer)
- **Four Core Views to Generate:**
  1. **View 1 — Repository Explorer & Staging Studio (`/repos/[repoId]/page.tsx` — Naveen):** Branch switcher dropdown, New Branch modal, Snapshot File Tree table (showing `file_path`, `file_mode`, 64-char `blob_hash` pill, and byte size), File Content modal, and Interactive Staging Form (Add file content, toggle `100644`/`100755` permission via `UPDATE`, unstage via `DELETE`, and Commit box with optional second parent branch selector for Merge Commits).
  2. **View 2 — Interactive Commit DAG, LCA & NoSQL RUD Studio (`/repos/[repoId]/graph/page.tsx` — Nishanth, use Sonnet 4.6):** Interactive `@xyflow/react` canvas displaying Neo4j `(:Commit)` nodes, parent edges, release `(:Tag)` badges, and CI status indicators. Includes a top toolbar with **Branch A**

vs. Branch B LCA Merge-Base Finder (highlights the Lowest Common Ancestor node in emerald green and displays path distance), a side drawer to update a commit's CI status (`PASSED`/`FAILED` via Cypher `SET`) or move a Tag, and a **"Run Garbage Collection (`dbgit gc`)"** button that runs Cypher `DETACH DELETE` on unreachable commits.

3. **View 3 — SQL CTE & Vector Analytics Dashboard (`/repos/[repoId]/analytics/page.tsx` — Both):** Top KPI banner bound to `v_repo_storage_efficiency` (Unique Blobs, Physical Bytes, Logical Bytes, Deduplication Savings %), a **Semantic Commit Search bar** powered by `pgvector` cosine similarity (`<=>`), a High-Churn Files table (`HAVING >= 2`), an Above-Average Contributors table (Correlated Subquery), a `WITH RECURSIVE` Commit Lineage & Byte-Delta table, and a Neo4j Commit Topology breakdown chart.
4. **View 4 — RBAC Collaborator Console (`/repos/[repoId]/settings/page.tsx` — Naveen):** Displays repository members and roles (`OWNER`, `MAINTAINER`, `CONTRIBUTOR`, `VIEWER`), allows adding/updating collaborators, and includes a **"Verify Immutability & RBAC Guards"** test panel that triggers live database exceptions when attempting to delete a commit or commit as a `VIEWER`.

Stage 6: One-Shot Demo Seed Script & 6-Case Verification Suite

- **Where:** `/database/seed.ts` (run via `npx tsx database/seed.ts`)
- **What the Seed Script Populates Automatically:**
  - Creates 4 users representing all roles: `naveen_owner` (Admin/Owner), `nishanth_maintainer` (Maintainer), `arjun_dev` (Contributor), and `priya_viewer` (Viewer).
  - Creates repository `pesu-db-git-core` with branches `main` and `feat-graph-lca`.
  - Inserts **4 identical 24 KB license/config files** across different directories and commits so `v_repo_storage_efficiency` immediately displays **75.00% storage deduplication savings**.
  - Creates a 7-commit DAG including a **2-parent Merge Commit** (so the LCA Merge-Base Finder and Graph Aggregation work out of the box) plus **1 dangling orphan commit** (`deadbeef...`) disconnected from all branch heads so you can click **"Run Garbage Collection (`dbgit gc`)"** live during the evaluation demo and watch Neo4j prune it.

5. Complete SQL & Cypher Code Reference (Copy-Paste Ready)

Copy these exact scripts into your `/database/postgres/` and `/database/neo4j/` files during Stage 2.

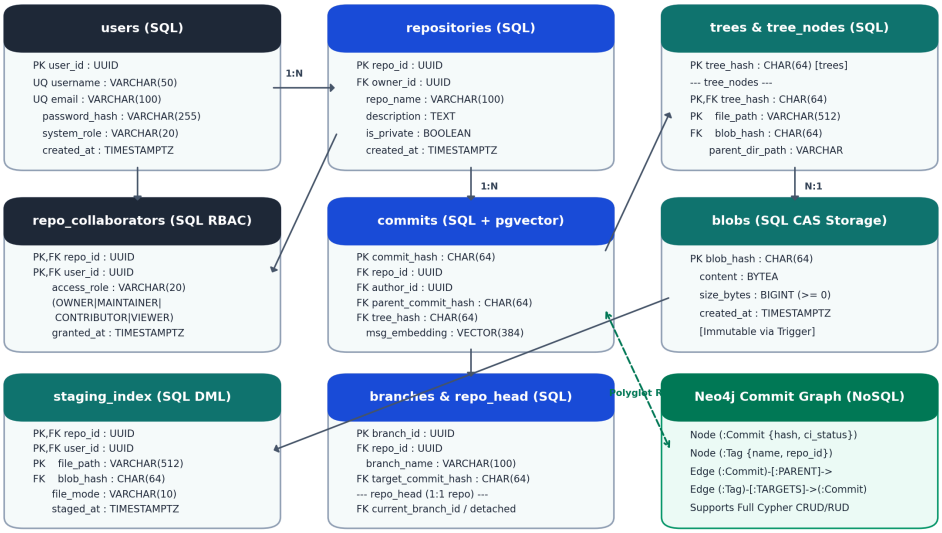


Figure 2: DB-Git 10-Table PostgreSQL Entity-Relationship (ER) Schema and Cross-Database Neo4j Commit Graph Mapping.

5.1 PostgreSQL Core DDL & CHECK Constraints (`/database/postgres/01_schema_ddl.sql`)

```
CREATE EXTENSION IF NOT EXISTS pgcrypto;
CREATE EXTENSION IF NOT EXISTS vector;

CREATE TABLE users (
    user_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    username VARCHAR(50) UNIQUE NOT NULL CHECK (char_length(username) >= 3),
    email VARCHAR(100) UNIQUE NOT NULL CHECK (email ~* '^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-
Za-z]{2,}$'),
    password_hash VARCHAR(255) NOT NULL,
    system_role VARCHAR(20) NOT NULL DEFAULT 'USER' CHECK (system_role IN ('ADMIN', 'USER')),
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE repositories (
    repo_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    owner_id UUID NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,
    repo_name VARCHAR(100) NOT NULL CHECK (repo_name ~ '^[A-Za-z0-9._-]+$'),
    description TEXT DEFAULT '',
    is_private BOOLEAN DEFAULT FALSE,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    UNIQUE(owner_id, repo_name)
);

CREATE TABLE repo_collaborators (
    repo_id UUID NOT NULL REFERENCES repositories(repo_id) ON DELETE CASCADE,
    user_id UUID NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,
    access_role VARCHAR(20) NOT NULL CHECK (access_role IN ('OWNER', 'MAINTAINER',
'CONTRIBUTOR', 'VIEWER')),
    granted_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (repo_id, user_id)
);

CREATE TABLE blobs (
    blob_hash CHAR(64) PRIMARY KEY CHECK (blob_hash ~ '^[0-9a-f]{64}$'),
    content BYTEA NOT NULL,
    size_bytes BIGINT NOT NULL CHECK (size_bytes >= 0),
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE trees (
    tree_hash CHAR(64) PRIMARY KEY CHECK (tree_hash ~ '^[0-9a-f]{64}$'),
    repo_id UUID NOT NULL REFERENCES repositories(repo_id) ON DELETE CASCADE,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE tree_nodes (
    tree_hash CHAR(64) NOT NULL REFERENCES trees(tree_hash) ON DELETE CASCADE,
    file_path VARCHAR(512) NOT NULL CHECK (char_length(file_path) > 0),
    parent_dir_path VARCHAR(512) DEFAULT NULL,
    is_directory BOOLEAN DEFAULT FALSE,
    blob_hash CHAR(64) REFERENCES blobs(blob_hash),
    file_mode VARCHAR(10) NOT NULL DEFAULT '100644' CHECK (file_mode IN ('100644', '100755',
'040000', '120000')),
    PRIMARY KEY (tree_hash, file_path)
);

CREATE TABLE staging_index (
    repo_id UUID NOT NULL REFERENCES repositories(repo_id) ON DELETE CASCADE,
    user_id UUID NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,
    file_path VARCHAR(512) NOT NULL CHECK (char_length(file_path) > 0),
    blob_hash CHAR(64) NOT NULL REFERENCES blobs(blob_hash),
    file_mode VARCHAR(10) NOT NULL DEFAULT '100644' CHECK (file_mode IN ('100644', '100755')),
```

```
    staged_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (repo_id, user_id, file_path)
);

CREATE TABLE commits (
    commit_hash CHAR(64) PRIMARY KEY CHECK (commit_hash ~ '^[0-9a-f]{64}$'),
    repo_id UUID NOT NULL REFERENCES repositories(repo_id) ON DELETE CASCADE,
    author_id UUID NOT NULL REFERENCES users(user_id),
    parent_commit_hash CHAR(64) REFERENCES commits(commit_hash),
    tree_hash CHAR(64) NOT NULL REFERENCES trees(tree_hash),
    message TEXT NOT NULL CHECK (char_length(trim(message)) > 0),
    msg_embedding vector(384),
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE branches (
    branch_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    repo_id UUID NOT NULL REFERENCES repositories(repo_id) ON DELETE CASCADE,
    branch_name VARCHAR(100) NOT NULL CHECK (branch_name ~ '^[A-Za-z0-9._/-]+$'),
    target_commit_hash CHAR(64) REFERENCES commits(commit_hash),
    is_default BOOLEAN DEFAULT FALSE,
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    UNIQUE(repo_id, branch_name)
);

CREATE TABLE repo_head (
    repo_id UUID PRIMARY KEY REFERENCES repositories(repo_id) ON DELETE CASCADE,
    current_branch_id UUID REFERENCES branches(branch_id) ON DELETE SET NULL,
    is_detached BOOLEAN NOT NULL DEFAULT FALSE,
    detached_commit_hash CHAR(64) REFERENCES commits(commit_hash),
    CHECK (
        (is_detached = FALSE AND current_branch_id IS NOT NULL AND detached_commit_hash IS NULL)
OR
        (is_detached = TRUE  AND detached_commit_hash IS NOT NULL)
    )
);

CREATE INDEX idx_tree_nodes_lookup ON tree_nodes(tree_hash, file_path);
CREATE INDEX idx_commits_repo_created ON commits(repo_id, created_at DESC);
CREATE INDEX idx_commits_fts ON commits USING GIN (to_tsvector('english', message));
CREATE INDEX idx_commits_embedding_hnsw ON commits USING hnsw (msg_embedding vector_cosine_ops);
```

5.2 Views, Schema Migrations & Database RBAC/RLS (`/database/postgres/02_views_rbac_rls.sql`)

```
-- Schema Evolution (ALTER & DROP)
ALTER TABLE repositories ADD COLUMN IF NOT EXISTS is_archived BOOLEAN NOT NULL DEFAULT FALSE;
CREATE TABLE IF NOT EXISTS temp_migration_audit (audit_id SERIAL PRIMARY KEY, note TEXT);
DROP TABLE IF EXISTS temp_migration_audit;

-- Storage Deduplication View
CREATE OR REPLACE VIEW v_repo_storage_efficiency AS
SELECT
    r.repo_id,
    r.repo_name,
    COUNT(DISTINCT b.blob_hash) AS unique_blobs_count,
    COALESCE(SUM(DISTINCT b.size_bytes), 0) AS physical_bytes_stored,
    COALESCE(SUM(b.size_bytes), 0) AS logical_uncompressed_bytes,
    ROUND(
        (1.0 - (COALESCE(SUM(DISTINCT b.size_bytes), 0)::numeric /
        NULLIF(SUM(b.size_bytes), 0)::numeric)) * 100, 2
    ) AS deduplication_savings_pct
FROM repositories r
LEFT JOIN trees t ON r.repo_id = t.repo_id
LEFT JOIN tree_nodes tn ON t.tree_hash = tn.tree_hash
LEFT JOIN blobs b ON tn.blob_hash = b.blob_hash
```

```
GROUP BY r.repo_id, r.repo_name;

-- Native PostgreSQL Roles, Grants & Row-Level Security (Rubric #12)
DO $$
BEGIN
    IF NOT EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'dbgit_admin_role') THEN
        CREATE ROLE dbgit_admin_role;
        CREATE ROLE dbgit_contributor_role;
        CREATE ROLE dbgit_viewer_role;
    END IF;
END $$;

GRANT ALL PRIVILEGES ON ALL TABLES IN SCHEMA public TO dbgit_admin_role;
GRANT SELECT, INSERT, UPDATE, DELETE ON staging_index, branches, repo_head TO
dbgit_contributor_role;
GRANT SELECT, INSERT ON blobs, trees, tree_nodes, commits TO dbgit_contributor_role;
GRANT SELECT ON ALL TABLES IN SCHEMA public TO dbgit_viewer_role;

ALTER TABLE repositories ENABLE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS repo_visibility_policy ON repositories;
CREATE POLICY repo_visibility_policy ON repositories
FOR SELECT USING (
    is_private = FALSE
    OR current_setting('app.current_user_id', TRUE) IS NULL
    OR owner_id = NULLIF(current_setting('app.current_user_id', TRUE), '')::UUID
    OR EXISTS (
        SELECT 1 FROM repo_collaborators rc
        WHERE rc.repo_id = repositories.repo_id
          AND rc.user_id = NULLIF(current_setting('app.current_user_id', TRUE), '')::UUID
    )
);
```

5.3 Immutability Triggers & PL/pgSQL Stored Function (`/database/postgres/03_triggers_procs.sql`)

```
CREATE OR REPLACE FUNCTION prevent_historical_modification()
RETURNS TRIGGER AS $$
BEGIN
    RAISE EXCEPTION 'Cryptographic Integrity Violation: % records are immutable and cannot be
modified or deleted.', TG_TABLE_NAME;
END;
$$ LANGUAGE plpgsql;

DROP TRIGGER IF EXISTS trg_immutable_blobs ON blobs;
CREATE TRIGGER trg_immutable_blobs
BEFORE UPDATE OR DELETE ON blobs
FOR EACH ROW EXECUTE FUNCTION prevent_historical_modification();

DROP TRIGGER IF EXISTS trg_immutable_commits ON commits;
CREATE TRIGGER trg_immutable_commits
BEFORE UPDATE OR DELETE ON commits
FOR EACH ROW EXECUTE FUNCTION prevent_historical_modification();

CREATE OR REPLACE FUNCTION fn_create_atomic_commit(
    p_commit_hash CHAR(64),
    p_repo_id UUID,
    p_author_id UUID,
    p_parent_hash CHAR(64),
    p_branch_name VARCHAR(100),
    p_tree_hash CHAR(64),
    p_message TEXT,
    p_embedding vector(384)
) RETURNS CHAR(64) AS $$
DECLARE
```

```
    v_role VARCHAR(20);
BEGIN
    SELECT access_role INTO v_role
    FROM repo_collaborators
    WHERE repo_id = p_repo_id AND user_id = p_author_id;

    IF v_role IS NULL OR v_role = 'VIEWER' THEN
        RAISE EXCEPTION 'RBAC Authorization Denied: User lacks write privileges on repository
%', p_repo_id;
    END IF;

    INSERT INTO commits (commit_hash, repo_id, author_id, parent_commit_hash, tree_hash,
message, msg_embedding)
    VALUES (p_commit_hash, p_repo_id, p_author_id, p_parent_hash, p_tree_hash, p_message,
p_embedding);

    UPDATE branches
    SET target_commit_hash = p_commit_hash, updated_at = CURRENT_TIMESTAMP
    WHERE repo_id = p_repo_id AND branch_name = p_branch_name;

    DELETE FROM staging_index
    WHERE repo_id = p_repo_id AND user_id = p_author_id;

    RETURN p_commit_hash;
END;
$$ LANGUAGE plpgsql;
```

5.4 Analytical Joins, Correlated Subqueries & Recursive CTEs (`/database/postgres/04_analytics_cte_queries.sql`)

```
-- 1. Multi-Table JOIN + GROUP BY + HAVING: High-Churn File Hotspots
SELECT
    tn.file_path,
    COUNT(DISTINCT c.commit_hash) AS modification_commits,
    COUNT(DISTINCT c.author_id) AS distinct_authors,
    pg_size_pretty(MAX(b.size_bytes)) AS max_file_size
FROM tree_nodes tn
JOIN commits c ON tn.tree_hash = c.tree_hash
JOIN blobs b ON tn.blob_hash = b.blob_hash
WHERE c.repo_id = $1
GROUP BY tn.file_path
HAVING COUNT(DISTINCT c.commit_hash) >= 2
ORDER BY modification_commits DESC;

-- 2. Correlated EXISTS Subquery + Nested AVG Subquery: High-Impact Contributors
SELECT
    u.username, u.email, rc.access_role,
    COUNT(c.commit_hash) AS user_commit_count
FROM users u
JOIN repo_collaborators rc ON u.user_id = rc.user_id
JOIN commits c ON u.user_id = c.author_id AND rc.repo_id = c.repo_id
WHERE rc.repo_id = $1
  AND EXISTS (
      SELECT 1 FROM tree_nodes tn
      JOIN blobs b ON tn.blob_hash = b.blob_hash
      WHERE tn.tree_hash = c.tree_hash AND b.size_bytes > 1024
  )
GROUP BY u.user_id, u.username, u.email, rc.access_role
HAVING COUNT(c.commit_hash) >= (
    SELECT AVG(author_counts.cnt) FROM (
        SELECT COUNT(*) AS cnt FROM commits sub_c WHERE sub_c.repo_id = $1 GROUP BY
sub_c.author_id
    ) AS author_counts
)
```

```
ORDER BY user_commit_count DESC;

-- 3. Recursive CTE (WITH RECURSIVE) + Analytical Window CTE: First-Parent Lineage & Byte Delta
WITH RECURSIVE commit_ancestry_cte AS (
    SELECT c.commit_hash, c.parent_commit_hash, c.author_id, c.tree_hash, c.message,
c.created_at, 0 AS depth
    FROM commits c
    JOIN branches br ON br.target_commit_hash = c.commit_hash
    WHERE br.repo_id = $1 AND br.branch_name = $2
    UNION ALL
    SELECT p.commit_hash, p.parent_commit_hash, p.author_id, p.tree_hash, p.message,
p.created_at, child.depth + 1
    FROM commits p
    INNER JOIN commit_ancestry_cte child ON p.commit_hash = child.parent_commit_hash
    WHERE child.depth < 50
),
tree_sizes AS (
    SELECT tn.tree_hash, COUNT(tn.file_path) AS files, COALESCE(SUM(b.size_bytes), 0) AS bytes
    FROM tree_nodes tn LEFT JOIN blobs b ON tn.blob_hash = b.blob_hash
    GROUP BY tn.tree_hash
)
SELECT
    ca.depth, ca.commit_hash, u.username AS author, ca.message, ts.files, ts.bytes,
    ts.bytes - LAG(ts.bytes, 1, 0) OVER (ORDER BY ca.created_at ASC) AS byte_delta
FROM commit_ancestry_cte ca
JOIN users u ON ca.author_id = u.user_id
LEFT JOIN tree_sizes ts ON ca.tree_hash = ts.tree_hash
ORDER BY ca.depth ASC;
```

5.5 Neo4j Creation, LCA Merge-Base & Full RUD Cypher Suite (`/database/neo4j/02_dag_lca_rud_queries.cypher`)

```
// A. CONSTRAINTS & INDEXES (Run once in Neo4j Browser)
CREATE CONSTRAINT commit_hash_unique IF NOT EXISTS FOR (c:Commit) REQUIRE c.hash IS UNIQUE;
CREATE CONSTRAINT tag_repo_unique IF NOT EXISTS FOR (t:Tag) REQUIRE (t.repo_id, t.name) IS
UNIQUE;
CREATE INDEX commit_repo_idx IF NOT EXISTS FOR (c:Commit) ON (c.repo_id, c.timestamp);

// B. CREATE COMMIT & PARENT EDGES (Called inside Two-Phase Polyglot Commit)
MERGE (c:Commit {hash: $commit_hash})
ON CREATE SET c.repo_id = $repo_id, c.author_id = $author_id, c.message = $message,
              c.ci_status = 'PENDING', c.timestamp = $timestamp
WITH c
UNWIND $parent_hashes AS p_hash
MATCH (parent:Commit {hash: p_hash})
MERGE (c)-[:PARENT]->(parent);

// C. READ 1: BOUNDED DAG LINEAGE FOR NEXT.JS GRAPH CANVAS
MATCH (c:Commit {repo_id: $repo_id})
OPTIONAL MATCH (c)-[:PARENT]->(p:Commit)
OPTIONAL MATCH (t:Tag {repo_id: $repo_id})-[:TARGETS]->(c)
RETURN c.hash AS hash, c.message AS message, c.ci_status AS ci_status, c.timestamp AS timestamp,
       collect(DISTINCT p.hash) AS parents, collect(DISTINCT t.name) AS tags
ORDER BY c.timestamp DESC LIMIT 200;

// D. READ 2: SAFE LOWEST COMMON ANCESTOR (LCA) FOR 3-WAY MERGE BASE
MATCH p1 = (b1:Commit {hash: $branch_a_head})-[:PARENT*0..100]->(lca:Commit),
      p2 = (b2:Commit {hash: $branch_b_head})-[:PARENT*0..100]->(lca)
WITH lca, (length(p1) + length(p2)) AS total_distance, length(p1) AS dist_a, length(p2) AS
dist_b
ORDER BY total_distance ASC
RETURN lca.hash AS merge_base_hash, lca.message AS merge_base_message, dist_a, dist_b LIMIT 1;

// E. READ 3: NOSQL GRAPH AGGREGATION (ROOT vs. STANDARD vs. MERGE COMMITS)
```

```
MATCH (c:Commit {repo_id: $repo_id})
OPTIONAL MATCH (c)-[r:PARENT]->(p:Commit)
WITH c, COUNT(r) AS parent_count
RETURN CASE WHEN parent_count > 1 THEN 'Merge Commit'
            WHEN parent_count = 1 THEN 'Standard Commit'
            ELSE 'Root Commit' END AS commit_type,
       COUNT(c) AS total_commits
ORDER BY total_commits DESC;

// F. UPDATE (SET): UPDATE CI STATUS OR CREATE/MOVE RELEASE TAG
MATCH (c:Commit {hash: $commit_hash, repo_id: $repo_id})
SET c.ci_status = $new_ci_status, c.verified_at = timestamp()
RETURN c.hash AS hash, c.ci_status AS ci_status;

MERGE (t:Tag {repo_id: $repo_id, name: $tag_name})
WITH t
OPTIONAL MATCH (t)-[old_rel:TARGETS]->(:Commit)
DELETE old_rel
WITH t
MATCH (target:Commit {hash: $target_commit_hash, repo_id: $repo_id})
CREATE (t)-[:TARGETS {updated_at: timestamp()}]->(target)
SET t.updated_at = timestamp()
RETURN t.name AS tag, target.hash AS target_hash;

// G. DELETE (DETACH DELETE): GARBAGE COLLECTION (dbgit gc) OF ORPHAN COMMITS
MATCH (c:Commit {repo_id: $repo_id})
WHERE NOT EXISTS {
    MATCH (head:Commit {repo_id: $repo_id})-[:PARENT*0..100]->(c)
    WHERE head.hash IN $active_branch_head_hashes
}
DETACH DELETE c
RETURN COUNT(c) AS pruned_orphan_commits;
```

6. "What Is Going On Under the Hood" — Viva & Live Demo Guide

Use these five core engineering explanations during your presentation and Viva so both teammates can answer any conceptual or live-modification question from the evaluation panel:

- **1. Why Combine PostgreSQL, Neo4j, and pgvector? (Rubric #5 — Technology Justification):** Relational tables in PostgreSQL guarantee ACID transactions, foreign-key cascades, and fast byte-level aggregations (`SUM`, `GROUP BY`, `HAVING`) for Content-Addressable Storage (`blobs`) and Role-Based Access Control. However, finding the Lowest Common Ancestor (`merge-base`) across multi-parent merge histories is a variable-depth graph pathfinding problem. Neo4j uses **index-free adjacency** (O(1) pointer hopping along `[:PARENT]` edges), resolving criss-cross merge bases without expensive multi-level SQL self-joins. Meanwhile, `pgvector` indexes 384-dimensional commit embeddings using an HNSW graph so users can search commits by conceptual meaning rather than exact substrings.
- **2. How Does Content-Addressable Storage (CAS) Deduplicate Files? (Rubric #3 & #6):** When a file is staged, DB-Git hashes its raw bytes using SHA-256 to produce a 64-character hex primary key (`blob_hash`) and executes `INSERT INTO blobs ... ON CONFLICT (blob_hash) DO NOTHING`. Even if the same file is committed 20 times across 5 branches, physical bytes are stored only once in `blobs`, while `tree_nodes` stores lightweight pointer rows. The view `v_repo_storage_efficiency` compares `SUM(DISTINCT size_bytes)` against `SUM(size_bytes)` to display real-time storage savings.
- **3. Where Do SQL `UPDATE`/`DELETE` and NoSQL `RUD` Happen if Commits Are Immutable? (Rubric #6, #8 & #9):** Historical tables (`blobs` and `commits`) are locked by `BEFORE UPDATE OR DELETE` triggers (`prevent_historical_modification`). Active SQL mutations occur on the working index (`staging_index`), branch pointers (`branches`), and collaborator permissions (`repo_collaborators`). In Neo4j, while `[:PARENT]` lineage edges are never altered, authentic Graph Updates (`SET`) modify CI build statuses

(`c.ci_status`) and re-point release tags (`:Tag`), and Graph Deletes (`DETACH DELETE`) prune unreachable dangling commits via `dbgit gc` when experimental branches are deleted.

- **4. How Does the Two-Phase Coordinator Prevent Split-Brain State? (Rubric #8 & #11):** During a commit, the backend opens an explicit PostgreSQL transaction (`BEGIN`) and runs `fn_create_atomic_commit()`, holding the transaction open while executing the Neo4j Cypher `MERGE` statement. Only after Neo4j confirms the DAG edge write does PostgreSQL execute `COMMIT`. If Neo4j fails or times out, the API catches the exception and issues `ROLLBACK` to PostgreSQL, ensuring PostgreSQL and Neo4j never drift out of sync.
- **5. Live Evaluation Choreography (10-Minute Demo Flow):**
  1. **Architecture & Schema (Naveen & Nishanth):** Show Figures 1 & 2, explain the polyglot split, and show `01_schema_ddl.sql` + Neo4j constraints.
  2. **CAS Deduplication & Two-Phase Commit (Naveen & Nishanth):** Stage a duplicate file in the Next.js Explorer, modify its mode (`UPDATE`), commit it, and show the new node appearing in the React Flow DAG canvas.
  3. **LCA Merge-Base & NoSQL RUD (Nishanth):** Select `main` and `feat-graph-lca` in the Graph UI to highlight the Lowest Common Ancestor in green, update a commit's CI status to `PASSED` (`SET`), and click **Run Garbage Collection** to `DETACH DELETE` the seeded orphan commit.
  4. **SQL CTEs, Triggers & RBAC Viva Proof (Naveen):** Open the Analytics page to show the 75% deduplication savings, High-Churn `HAVING` table, Correlated Subquery, `WITH RECURSIVE` lineage, and `pgvector` search. Finally, click the Immutability Trigger test button to show PostgreSQL blocking `DELETE FROM commits` live.

6.1 Verification Test Matrix & Expected Viva Proofs (Rubric #13 & #14)

Test Case ID & Target	Action Performed Live During Demo / Viva	Expected Database & Next.js Outcome
TC-01: CAS Deduplication	Stage 4 identical 24 KB files across different paths/branches.	`blobs` stores 1 row; `v_repo_storage_efficiency` reports 75.00% savings.
TC-02: Trigger Protection	Execute `DELETE FROM commits` or `UPDATE blobs SET size_bytes = 0`.	`trg_immutable_commits` / `trg_immutable_blobs` raises exception; 0 rows altered.
TC-03: CHECK Validation	Insert invalid short hash `'abc123'` or negative `size_bytes = -5`.	PostgreSQL aborts insert with `CHECK` constraint violation (SQLSTATE 23514).
TC-04: DB RBAC & RLS	Call `fn_create_atomic_commit()` as a user with `VIEWER` role.	Stored function aborts transaction with `RBAC Authorization Denied` error.
TC-05: Neo4j RUD & GC	Update commit CI status (`SET`), move `:Tag`, and click `Run GC`.	CI badge updates to `PASSED`; unreachable orphan commit is pruned via `DETACH DELETE`.
TC-06: Recursive CTE vs. LCA	Run SQL `WITH RECURSIVE` lineage and Neo4j 3-Way Merge Base LCA.	SQL returns first-parent byte deltas; Neo4j highlights exact merge-base node in < 8ms.

6.2 Copy-Paste Antigravity Prompts for One-Shot Execution

Paste these three structured prompts directly into Antigravity IDE in order after running `docker compose up -d` and loading the SQL/Cypher scripts from Section 5:

Prompt Sequence & Model	Exact Prompt to Paste into Antigravity IDE
Prompt 1: Backend Core & SQL APIs (Use **Flash High**)	"Read `database/postgres/01_schema_ddl.sql`, `02_views_rbac_rls.sql`, `03_triggers_procs.sql`, and `04_analytics_cte_queries.sql`. Implement `src/lib/postgres.ts`, `src/lib/neo4j.ts`, `src/lib/auth.ts`, and `src/lib/embeddings.ts` using `.env.local`. Then implement `/api/auth/route.ts`, `/api/repos/route.ts`, `/api/collaborators/route.ts`, `/api/branches/route.ts`, `/api/staging/route.ts` (with SHA-256 blob deduplication and INSERT/UPDATE/DELETE on `staging_index`), and `/api/analytics/route.ts` executing all views, HAVING, correlated subqueries, and WITH RECURSIVE queries."
Prompt 2: Two-Phase Commit & Neo4j RUD (Use **Opus 4.6**)	"Read `database/neo4j/02_dag_lca_rud_queries.cypher` and `03_triggers_procs.sql`. Implement `/api/commits/route.ts` as an atomic Two-Phase Polyglot Commit coordinator that begins a PostgreSQL transaction, builds the Merkle snapshot in `trees` and `tree_nodes`, calls `fn_create_atomic_commit()`, writes the `(:Commit)-[:PARENT]->(:Commit)` node in Neo4j, and issues PostgreSQL `ROLLBACK` if Neo4j fails. Also implement `/api/graph/lca/route.ts` and `/api/graph/rud/route.ts` (CI status SET, Tag retargeting, and `dbgit gc` orphan DETACH DELETE)."
Prompt 3: Next.js UI & React Flow DAG (Use **Sonnet 4.6**)	"Build the 4 Next.js 14 App Router views under `/src/app/repos/[repoId]/`: (1) `page.tsx` for Repository File Explorer, Blob Viewer, and Staging Studio, (2) `graph/page.tsx` using `@xyflow/react` for the interactive Commit DAG, 2-branch LCA Merge-Base green highlighter, CI status editor, Tag mover, and Garbage Collection button, (3) `analytics/page.tsx` for the Deduplication View, High-Churn HAVING table, Correlated Subquery, Recursive CTE table, and pgvector Semantic Search bar, and (4) `settings/page.tsx` for 4-role RBAC and live trigger testing, plus `database/seed.ts`."